TESTGEN AI

AI-Powered • Multi-Language • Levels-wise Testing • CI/CD Ready

PROJECT DOC

01 Problem Statement

Software testing is a critical phase in the development lifecycle, ensuring code reliability, security, and performance. However, writing comprehensive test cases manually is incredibly time-consuming, prone to human error, and often neglects edge cases.

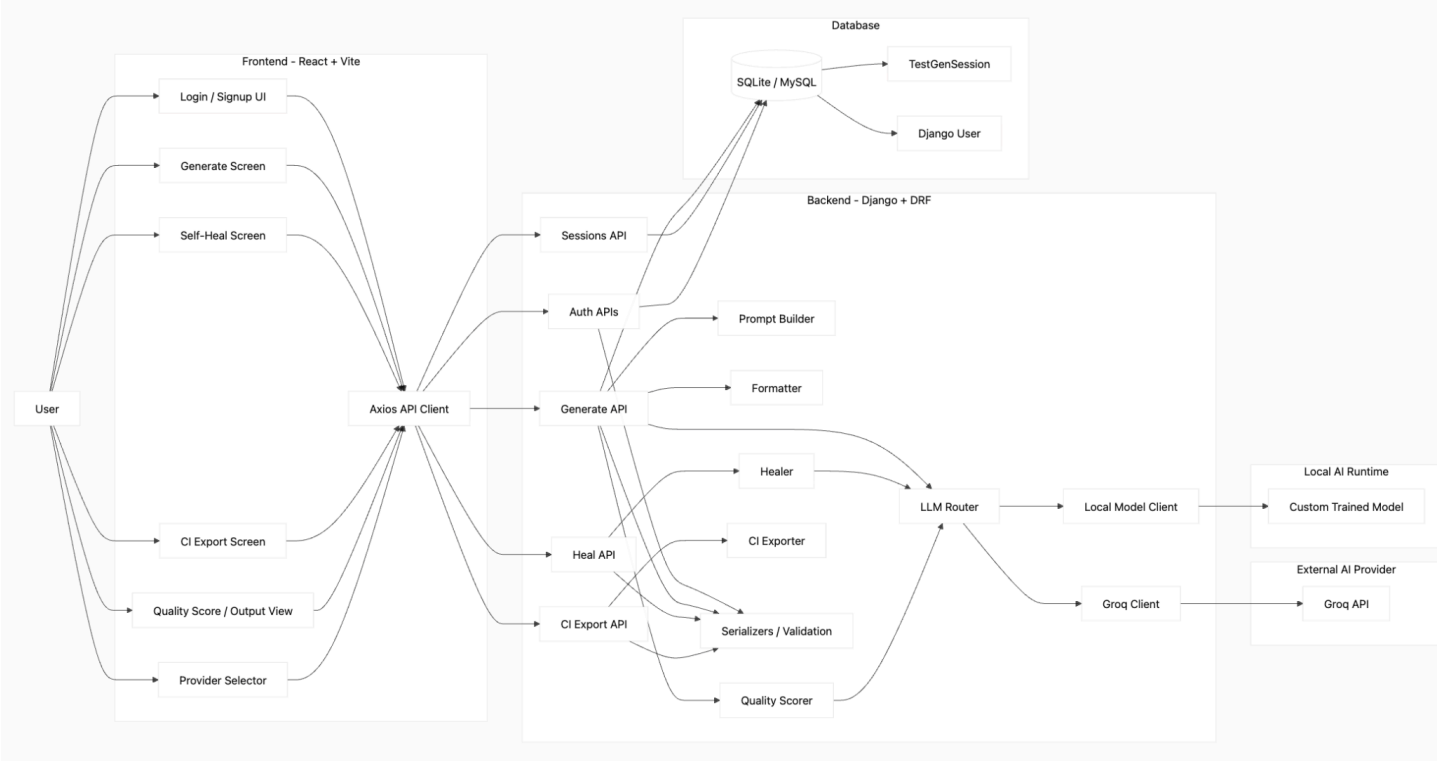
> **The Core Challenge**
>
> *Developers frequently spend a significant portion of their time crafting unit tests instead of focusing on core feature development. There is a pressing need for an automated, intelligent solution that can instantly analyze code, functional requirements, or API endpoints — generating robust, executable test cases while guaranteeing high code coverage. Our main challenge till now was in creating local model and training the model with parameters.*

02 Implementation Approach

We developed an AI-powered Automatic Testcase Generator structured into fourth distinct stages to ensure iterative validation and robust development. The system leverages Large Language Models (LLMs) and parsing algorithms to understand code context, dependencies, and behavioral intent.

High Level Design (HLD)

Frontend Layer	A React-based interface offering an intuitive and dynamic user experience.
Backend API Layer	A robust backend that securely processes user inputs and interfaces with AI models to return structured test files.
Model Layer	Flexible architecture supporting both high-speed cloud inference (Groq) and secure local model Qwen 1.5B to accommodate different enterprise privacy requirements.



03 Stage-Wise Development

1 Foundation & Proof of Concept

Focus: Requirement Analysis · Single Language · Basic Mathematical Testing

- **Requirement Analysis:** Structured the core input-to-output pipeline, defining how the system processes raw text/code and translates it into testing frameworks.
- **Single Programming Language (Python):** Restricted the initial MVP scope to Python, allowing deep refinement of prompt engineering for pytest conventions.
- **Mathematical Problem Testing:** Rigorously stress-tested on algorithmic and mathematical functions — verifying boundary value behaviors (negative inputs, zero, large integers) and edge-case assertion generation.

2 Feature Expansion & System Scalability

Focus: Authentication · Input Modalities · Multi-Language Frameworks · Frontend

- **Sign-in / Sign-up:** Implemented secure user authentication to allow developers to manage sessions seamlessly.
- **Diverse Input Modalities:** Integrated a code snippet editor, file upload support, and user story/API endpoint acceptance for flexible test generation.
- **Testing Methodologies:** Users can toggle between Black Box (input/output behavior) and White Box (internal state and logic) generation modes.
- **Multi-Language Support:** Scaled the generation engine to support Pytest (Python), JUnit (Java), and Jest (JavaScript).

- **Quality Score:** Introduced an analysis layer evaluating generated tests on assertion density and estimated coverage.
- **Basic Frontend:** Deployed the first responsive web layer connecting backend capabilities into a unified user interface.

3

Advanced Automation & Enterprise Readiness

Focus: Self-Healing Tests · Testing Level · CI Export · Enhanced UI

- **Self-Heal Test Capability:** The flagship 'Heal Panel' ingests failing test output and compiler stack traces, auto-diagnosing flaws and regenerating corrected test snippets.
- **Testing Level**: It allows testing based on level like Unit Testing, Integration Testing, UAT (Acceptance) testing and System testing.
- **CI Export:** Automatically generates configured YAML workflow files, enabling instant export to GitHub Actions pipelines.
- **Enhanced UI (Dark Minimal Theme):** Completely redesigned with a dark, modern, premium aesthetic — smooth transitions, dynamic interactions, and a developer-friendly layout.

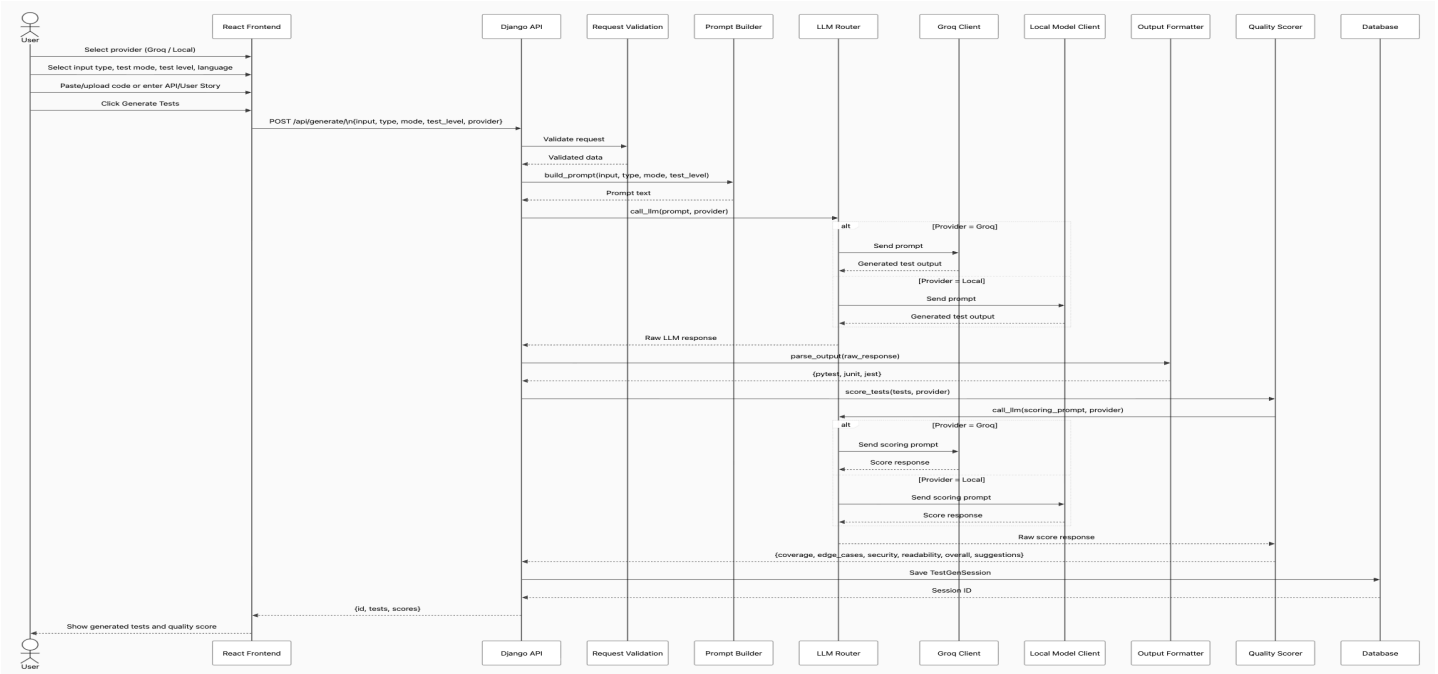
4

Final Stage (Will be working next)

Focus: Integration of created local model with addition to API · Functionality to switch between using API or local model · Deployment

- **Local AI Model Integration:** Backend fully decoupled to support localized inference, guaranteeing absolute data privacy and zero code leakage to external APIs.
- **Toggle Button**
- **Deployment**

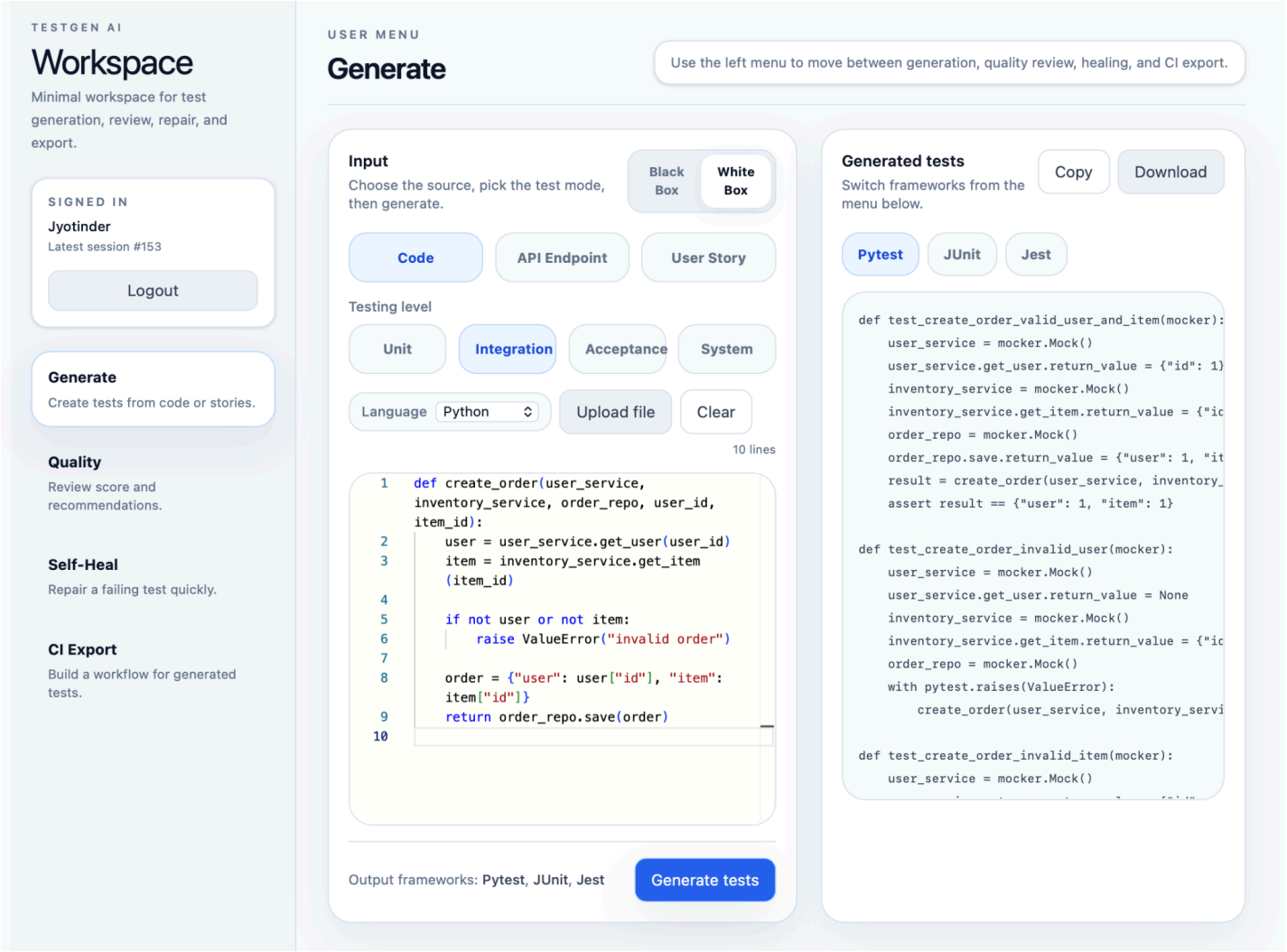
Sequence Diagram



04 Feature Summary

Feature	Description
AI Code Analysis	LLM-powered parsing of code context, dependencies, and behavioral intent
Multi-Language	Pytest · JUnit · Jest support out of the box
Input Flexibility	Code paste, file upload, user stories, API endpoint specs
Test Modes	Black Box (behavior) and White Box (logic) generation
Quality Score	Assertion density and coverage estimation per test suite
Self-Healing	Auto-diagnosis and correction of failing tests from stack traces
CI/CD Export	One-click GitHub Actions YAML configuration export
Local AI Models	On-premise inference for enterprise data privacy compliance

UI (Screenshot)



05 Team Members (From IIIT Dharwad)

- Alok Kumar Das - Team Lead
- Deependra Kumar Singh
- Jyotinder Yadav
- Dharmendra Gora

From Proof of Concept to Enterprise-Grade Automation

The TESTGEN AI transforms how development teams approach software quality — eliminating manual effort, accelerating coverage, and integrating seamlessly into modern CI/CD pipelines.